

We need to get rid of these old bullets, or the game will slow down from doing so much unnecessary work. To do this, we need to detect when the bottom value of a bullet's rect has a value of 0, which indicates the bullet has passed off the top of the screen:

alien
_invasion.py

```
def run_game(self):
    """Start the main loop for the game."""
    while True:
        self._check_events()
        self.ship.update()
        self.bullets.update()

        # Get rid of bullets that have disappeared.
        ❶ for bullet in self.bullets.copy():
            ❷ if bullet.rect.bottom <= 0:
                ❸ self.bullets.remove(bullet)
            ❹ print(len(self.bullets))

        self._update_screen()
        self.clock.tick(60)
```

When you use a for loop with a list (or a group in Pygame), Python expects that the list will stay the same length as long as the loop is running. That means you can't remove items from a list or group within a for loop, so we have to loop over a copy of the group. We use the `copy()` method to set up the for loop ❶, which leaves us free to modify the original bullets group inside the loop. We check each bullet to see whether it has disappeared off the top of the screen ❷. If it has, we remove it from bullets ❸. We insert a `print()` call to show how many bullets currently exist in the game and verify they're being deleted when they reach the top of the screen ❹.

If this code works correctly, we can watch the terminal output while firing bullets and see that the number of bullets decreases to zero after each series of bullets has cleared the top of the screen. After you run the game and verify that bullets are being deleted properly, remove the `print()` call. If you leave it in, the game will slow down significantly because it takes more time to write output to the terminal than it does to draw graphics to the game window.

Limiting the Number of Bullets

Many shooting games limit the number of bullets a player can have on the screen at one time; doing so encourages players to shoot accurately. We'll do the same in *Alien Invasion*.

First, store the number of bullets allowed in *settings.py*:

settings.py

```
# Bullet settings
--snip--
self.bullet_color = (60, 60, 60)
self.bullets_allowed = 3
```